

We were told by a TA that we don't need to provide a write-up for the experiments and just do the take-home exercise.

Chris Jiang 400322193

Sohel Saini 400327615

Exercise:

We began by using the code from Experiment 4 as the foundation for completing the take-home exercise. We split each of the counters into their respective digits for example we split the frame counter into its ones, tens, and hundreds. We did the same thing with the PP counter. First, we initialize variables `number_press`, `current_digit`, and `prev_digit`. Then check to see if only a number were pressed once a number was pressed it set the flag `number_press` to 1, stored the `ps2_code` into the `ps2_reg` and then put the `current_digit` into the `prev_digit`. We have a case block to map the `ps2_code` into the corresponding decimal number and set that to the `current_digit`. If a key that is not 0-9 is pressed nothing happens as `number_press` remains set to 0. Next, we created an `always_ff` block to track when the number keys are pressed on the keyboard and keep count of how many times they were pressed essentially the key pressed counter. Next, we have another `always_ff` block that increments a counter called `PP_count` every time a numeric key is pressed and stores the previous value of the counter in `prev_pp`. This block also contains logic for when the counter reaches 99 it will reset it back to 0 creating a counter that goes for 0-99 infinitely. An `always_ff` was created to detect the falling edge of the `VGA_VSYNC_0` signal the logic is for a falling edge it needs to go from high to low so the previous value of the signal `v_sync_prev` has to be 1 and the current value of `VGA_VSYNC_0` must be 0. Next, we made a block for the `FFF_count`. If no key is pressed `FFF_count` is 0. If a number key is pressed the counter is also set to 0. The counter only begins to increment on the falling edge of the clock and goes until 999. The next block of code handles the maximum value of `FFF_count`, `prev_pp`, and `prev_digit`. `max_FFF` stores the maximum value `FFF_count` and it's 10 bits. `max_PP` stores the `prev_PP` when `max_FFF` occurs. `Max_digit` stores the value of `prev_digit` when `max_FFF` occurs and it's 1 bit. When `number_press` is 1 it will check if `FFF_count` exceeds the stored `max_FFF` and if it is we will replace it. `max_PP` gets updated by incrementing 1 to `prev_PP` and the max digit was just the `prev_digit`. The last block of code we have is to display the characters on the screen. When a key is pressed it will check where on the screen we should like the text to be displayed. We then have a case statement to display the default message when a key is pressed and then the max message displays when two keys have been pressed. Now talking about register usage each signal had bits allocated to be able to hold the data needed. Our register usage was 145 registers and that was due to how many bits we allocated to all of our signals. If we wanted to reduce register usage we would need to minimize the number of bits allocated to the signals. There is a direct correlation of bits to registers the more bits we use the more registers we will use.

7	✓ Total registers*	145 / 117,053 (< 1 %)
1	-- Dedicated logic registers	145 / 114,480 (< 1 %)
2	-- I/O registers	0 / 2,573 (0 %)
0		